

寄存器和双堆栈指针

ARM Cortex-M通过三个寄存器实现了双堆栈指针设计

双堆栈指针实现了：

1. 内核与进程的隔离
2. 降低了切换任务的开销
3. 支持特权/非特权

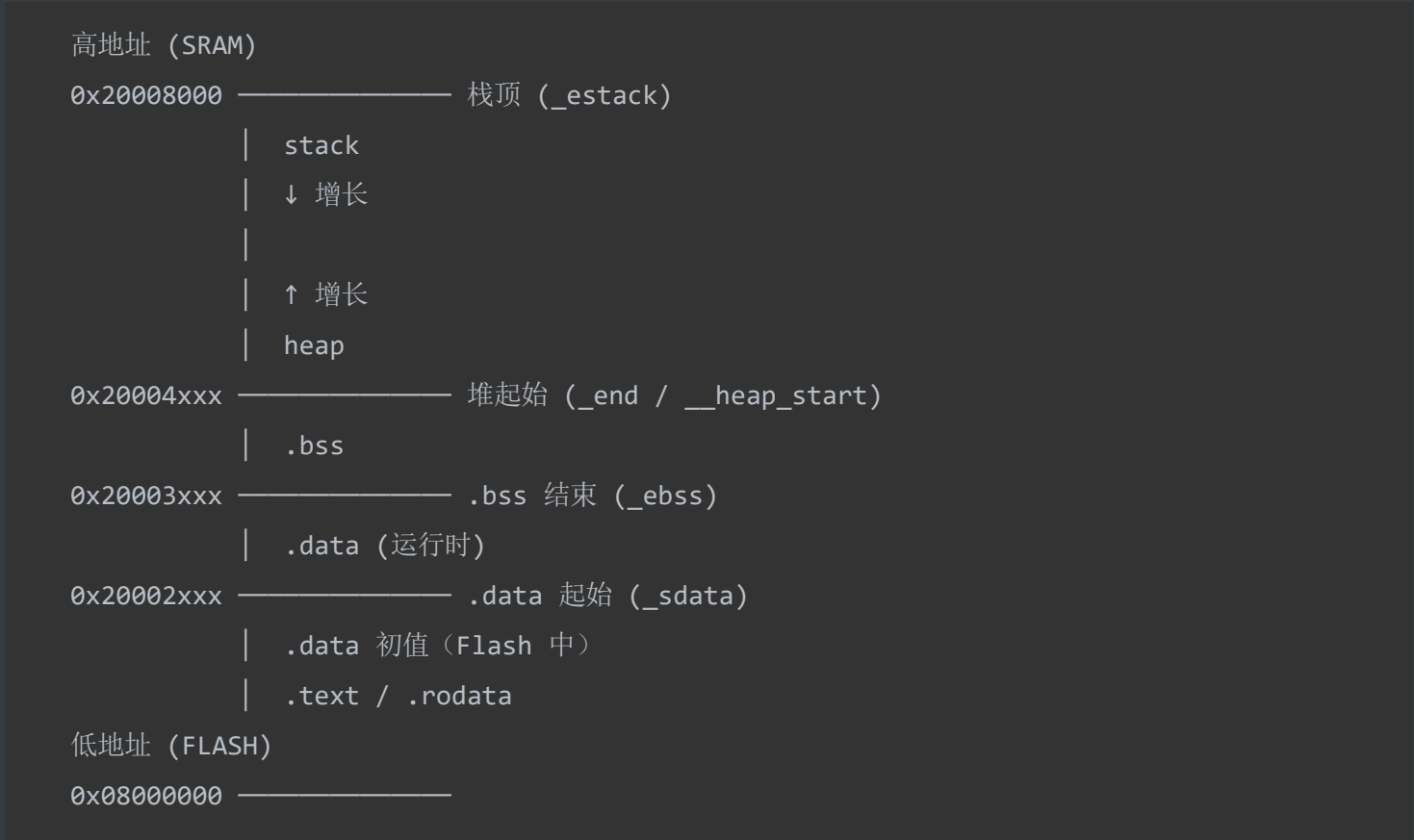
接下来从寄存器开始介绍

寄存器

Arm内核的寄存器

```
r0 # 通用寄存器 低寄存器 有一些16bit指令 只能使用低组寄存器
r1 # 通用寄存器 低寄存器
r2 # 通用寄存器 低寄存器
r3 # 通用寄存器 低寄存器
r4 # 通用寄存器 低寄存器
r5 # 通用寄存器 低寄存器
r6 # 通用寄存器 低寄存器
r7 # 通用寄存器 低寄存器
r8 # 通用寄存器 高寄存器
r9 # 通用寄存器 高寄存器
r10 # 通用寄存器 高寄存器
r11 # 通用寄存器 高寄存器
r12 # 通用寄存器 高寄存器
r13(SP) # 栈指针
r14(LR) # 连接寄存器
r15(PC) # 程序计数器
# 除此之外，还有一些特殊寄存器，比如CONTROL寄存器
```

堆栈模型



问题：为什么堆栈要设计成一个向下增长、一个向上增长，两者对向增长的样子？

回答：因为SRAM的大小是固定的，堆栈如果设计成同向的，就必须设计一个安全的分界线，而且在使用堆栈的时候还要注意不能越过分界线，操作起来比较麻烦，同时还会产生内存碎片的问题，因此干脆就设计成一个从底部向上增长，一个从头上向下增长的样子

栈寄存器

Arm的栈寄存器SP实际上是由两个不同的寄存器实现的，分别是MSP（main stack pointer）和PSP（process stack pointer）

但是CPU同时只能使用两者中的一个，Arm设计了一个特殊的寄存器CONTROL用于指明具体使用哪一个SP

通过之前的文章可知，进行复位的时候，使用的是MSP，一开始给MSP赋值为0x2000 8000

入栈出栈操作

```

# 汇编代码

.syntax unified
.cpu cortex-m3
.thumb

.section .isr_vector, "a"
.word 0x20008000
.word boot_code

.global boot_code
.type boot_code, %function

boot_code:
    mov r1, #0x01
    push {r1}

    mov r2, #0x02
    push {r1, r2}

    pop {r3, r4}

    # 切换到PSP
    ldr r0, =0x20000400
    msr psp, r0 # 给psp指针赋值

    mov r0, #2
    msr control, r0 # 通过操作control寄存器切换到PSP
    # 之后再进行push pop操作，就以0x2000 0400为基准了

```

分析一下push和pop操作之后栈里的情况：

push {r1} 之后

0x20008000

0x20007ffc 0x01

push {r1, r2} 之后

0x20008000

0x20007ffc 0x01

0x20007ff8 0x02

0x20007ff4 0x01

出栈的时候后入栈的先出

pop {r3, r4} 之后

r3 = [0x20007ffc] = 0x01

r4 = [0x20007ff8] = 0x02